

EvoLab — A Digital Evolution & Phylogenetics Laboratory

OOP Project | Team of 4

The One-Line Pitch

We simulate a population of digital organisms evolving under real diploid genetics and natural selection, record their true ancestry as it happens, then try to reconstruct that ancestry blind from genome data alone — and measure how well the reconstruction matches the truth.

That last part is what makes this a research project and not just a simulation: we're not only building an evolving ecosystem, we're using it to run a real experiment in molecular evolution — *how accurately can phylogenetics recover the truth, and when does it break?*

Why This Project

Deep biology, not bio-flavored code. Real diploid Mendelian genetics, real meiosis, real population genetics diagnostics (Hardy-Weinberg, drift vs. selection). This isn't organisms-with-numbers — it's an actual genetic model.

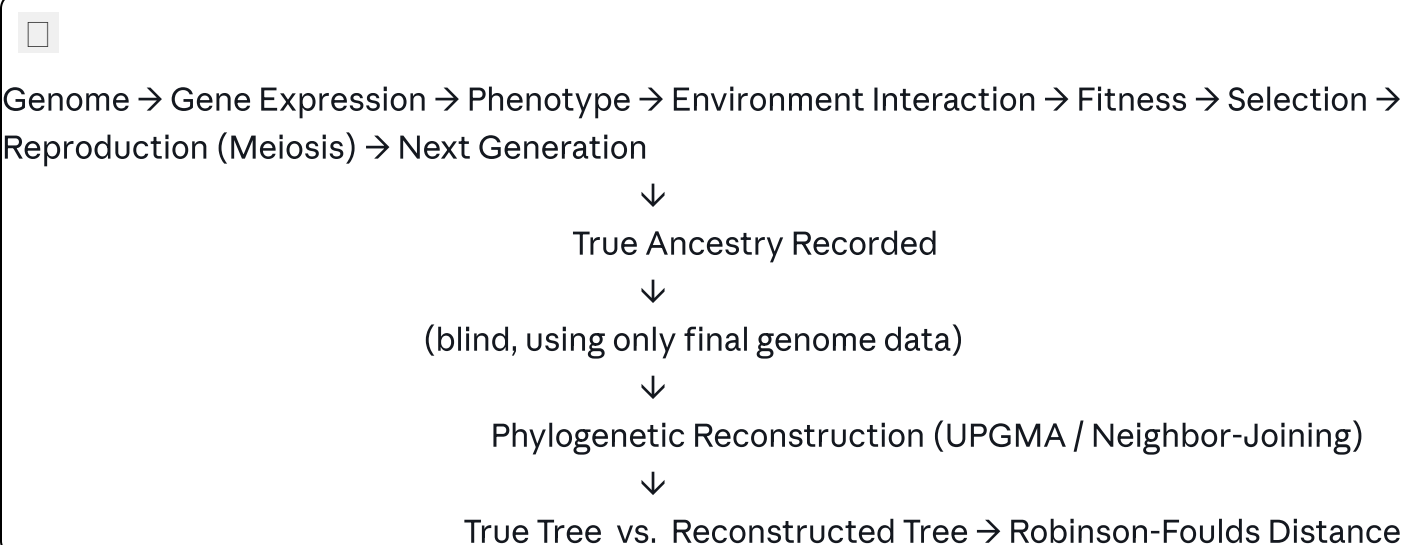
A real research question with a quantitative answer, not just a cool demo.

The best visual payoff of anything we considered — a living ecosystem next to a phylogenetic tree building itself in real time, split-screen.

The strongest legitimate OOP surface area — six design patterns, each one solving a problem the domain actually creates, not retrofitted for a rubric.

Fully bio-leaning without a single external dependency on a specific target program — it stands on its own as a serious project, and happens to read very well for anyone interested in computational biology, evolutionary genetics, or bioinformatics.

The Core Idea, End to End



We watch a population evolve, then ask our own algorithm to guess how it evolved — and grade it.

The Genetics Core (this is what makes it "real")

Every organism carries a **diploid genome** — two alleles per gene, one inherited from each parent, exactly like real inheritance.

Gene — a locus, with an expression rule attached (dominant/recessive, additive/polygenic, or codominant)

Allele — a specific variant at that locus, carrying an effect value

Genotype — the organism's actual pair of alleles at every locus

Phenotype — computed live from genotype via each gene's expression rule

Reproduction isn't "average the parents' stats." It's **simulated meiosis**: each parent contributes one randomly segregated allele per locus (independent assortment), with a chance of mutation on each allele copy, and — if we get to it — crossover/linkage between genes on the same "chromosome."

Because we track real allele frequencies generation over generation, we get genuine population genetics phenomena for free:

Genetic drift — random allele frequency wander, more visible in small populations

Selection — directional frequency shift toward fitter alleles

Hardy-Weinberg deviation — we compute the expected genotype ratios under equilibrium each generation and plot our actual population's deviation from it — a real, standard population-genetics diagnostic

Fitness is never assigned by us — it **emerges** from phenotype meeting environment (e.g., a coat-color gene's match to terrain color determines camouflage; a metabolism gene trades efficiency against speed). This is what makes selection feel earned instead of scripted.

The Research Question

Under our modeled genetic architecture, how accurately does distance-based phylogenetic reconstruction (UPGMA vs. Neighbor-Joining) recover true evolutionary ancestry — as a function of mutation rate, population size, and time since divergence — and where does reconstruction break down?

We measure this with **Robinson-Foulds distance** between the true recorded ancestry tree and the blindly reconstructed tree, across a set of controlled experiments.

The Three Experiments

Mutation rate sweep — low vs. medium vs. high mutation, compare genetic diversity, extinction rate, and reconstruction accuracy

Geographic isolation — split the population into two environments with zero gene flow, run for many generations, then test whether reconstruction correctly identifies the divergence (this is a simplified but genuine model of allopatric speciation)

Environmental shift — sudden change mid-run (e.g., resource distribution or terrain shift), observe the selection pressure spike and resulting lineage response

Each experiment produces a real plot and a real number. That's the difference between a demo and a study.

The Visual Experience

Split-screen, always live, always in sync:

Left: the ecosystem. Organisms rendered with phenotype-driven appearance (color from the coat-color gene, size from the size gene) moving across a terrain-colored grid. You can visually watch camouflage-

driven selection happen — mismatched organisms get picked off first.

Right: the phylogenetic tree, radial layout, branching live as lineages diverge — the same visual language as real published "tree of life" figures.

Supporting panels:

Live allele frequency plot (one line per allele per gene, tracked across generations) — the single most recognizable population-genetics visual there is

Genotype distribution histogram vs. Hardy-Weinberg expected ratios

Fitness landscape heatmap, with the population's trait cloud visibly climbing toward local optima over time

At experiment's end: **true ancestry tree vs. reconstructed tree, side by side**, with the Robinson-Foulds score displayed

This is designed to be the kind of thing you'd record a 20-second clip of and actually want to put on a portfolio site.

OOP Architecture



Organism

- └─ Genotype (diploid: Gene → Allele pairs)
- └─ Phenotype (derived via ExpressionRule per gene)
- └─ LifeStage (State: Juvenile → Adult → Senescent)
- └─ Behavior (Strategy: Foraging, Fleeing)

Genome

- └─ Gene (locus + ExpressionRule)
- └─ Allele (variant + effect value)

EvolutionEngine

- └─ ReproductionStrategy (Strategy: meiosis — segregation, crossover, mutation)
- └─ FitnessFunction (Composite: per-trait fitness contributions summed)
- └─ PopulationGeneticsTracker (allele frequencies, HW deviation, drift/selection diagnostics)

AncestryRecorder

- └─ the true parent-child lineage graph, recorded as it happens — our ground truth

Phylogenetics

- └─ DistanceMatrix (built from genome similarity)
- └─ TreeBuildingAlgorithm (Strategy: UPGMA, Neighbor-Joining)
- └─ TreeComparator (reconstructed vs. AncestryRecorder ground truth — Robinson-Foulds distance)

Environment

- └─ Region (Composite: terrain + resource properties)
- └─ ExperimentController (Command: pause / rewind / speed / trigger climate shift / trigger isolation)

Visualization

- └─ EcosystemRenderer
- └─ PhylogeneticTreeRenderer (dual mode: true vs. inferred)
- └─ StatsPanel (allele frequency plot, HW histogram, fitness landscape)

Patterns in play — and why each one is earned, not forced

Pattern	Where	Why it's genuinely needed
Strategy	<code>ExpressionRule</code> , <code>ReproductionStrategy</code> , <code>Behavior</code> , <code>TreeBuildingAlgorithm</code>	Multiple interchangeable algorithms behind one interface — lets us swap and benchmark UPGMA vs. NJ, or dominant vs. additive expression, without touching calling code
Composite	<code>FitnessFunction</code> , <code>Region</code> / <code>Environment</code>	Fitness is naturally a sum of independent trait contributions; environment is naturally a grid of regions — the domain shape <i>is</i> Composite
Observer	Simulation engine → Visualization	renderer, and stats panel all listen independently, fully decoupled from simulation logic
State	<code>LifeStage</code>	Organism behavior genuinely changes by life stage (juvenile can't reproduce, senescent has declining fitness) — a textbook State use case
Command	<code>ExperimentController</code>	Pause/rewind/speed-up/trigger-intervention are literally executable, undoable actions — Command is the natural fit, not a stretch
Factory	Structure/organism initialization	Building starting populations and parsing experiment configs cleanly

Suggested Team Split (4 people)

Genetics Core — `Gene`, `Allele`, `Genotype`, `Phenotype`, `ExpressionRule`, `ReproductionStrategy` (meiosis, mutation, crossover), `AncestryRecorder`

Evolution + Environment — `EvolutionEngine`, `FitnessFunction`, `Environment` / `Region`, `ExperimentController` (Command pattern, the three experiments)

Phylogenetics + Analysis — `DistanceMatrix`, `UPGMA` / `NeighborJoining`, `TreeComparator` (Robinson-Foulds), `PopulationGeneticsTracker` (allele frequency, HW deviation)

Visualization + Report — dual ecosystem/tree renderer, all stats panels, live animation, and owns final report assembly across all four sections

Roles 1 and 2 are the dependency backbone — agree on the `Organism` / `Genotype` / `Phenotype` data contract as a team on day one, then everyone can build in parallel against that shared shape without blocking each other.

What We're Explicitly Not Doing (to stay finishable)

No predators, no multiple simultaneous environmental pressure axes — one or two environmental factors (terrain/camouflage + one resource axis) is enough depth without burying the genetics under world-building

No promise that visually distinct "species" will emerge on cue — that's a possible outcome of a well-tuned run, not something to script or guarantee for the demo. The reliable fallback demo material is the allele-frequency plots and the true-vs-reconstructed tree comparison, which will show something real regardless of exact outcome

Crossover/linkage between genes is a nice-to-have refinement, not required for the core loop to work

Stretch Goals (only after the core loop runs end to end)

Gene linkage / crossover between loci on the same "chromosome"

Sexual selection (mate choice as an additional selection pressure)

A third tree-building method (Maximum Parsimony) added to the comparison

Interactive god-mode interventions beyond the three core experiments

Next step: turn this into a week-by-week build guide once everyone's read this and we've agreed on scope.